AI for Economic Research: Dynamic Models, Language, and Agents
Lecture 8.2: RAG — Chunking and Embeddings

Zhigang Feng

Spring 2026

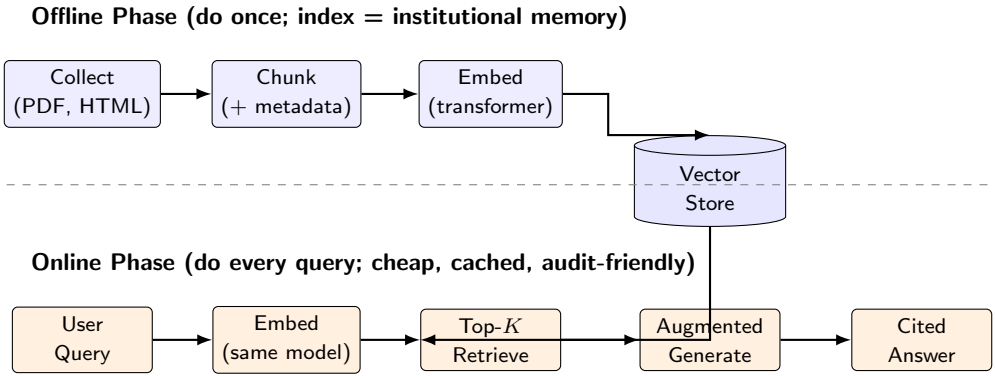
Topic 8.2 Agenda

1. **RAG — The Principled Solution**
2. **Chunking**
3. **Embeddings** — dense vector representations

Arc: T1 Prompting wall → **T2 Chunking & embeddings** → *T3 Vector & graph retrieval* → *T4a Augmented generation* → *T4b Demo, failures, agentic bridge*

RAG — The Principled Solution

The Institutional-Memory Picture: Offline Index vs Online Query



- **Offline (blue)** is the costly, one-time work — build the index over the FOMC / NBER / 10-K archive.
- **Online (orange)** is the cheap, repeated work — one query embeds once, retrieves the top-K, ships only that to the LLM.
- **The separation is the point**: the same index supports thousands of queries over months without re-paying the ingestion bill.

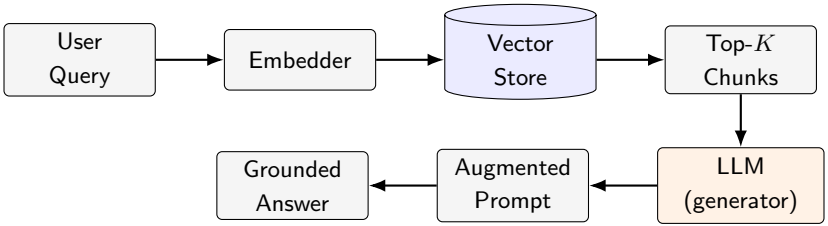
The Key Insight

- Don't paste the whole corpus into every prompt.
- Instead:
  1. **Pre-process documents once.** Split them into chunks, convert each chunk to a numerical fingerprint (an *embedding*), store everything in a database.
  2. **At query time, fetch only what's relevant.** Convert the user's question to an embedding, look up the closest chunks in the database, paste *those few chunks* into the prompt.
  3. **Let the LLM answer from the retrieved evidence**, citing which chunks it used.
- This is a **two-phase architecture:**
  - *Offline:* index everything once.
  - *Online:* retrieve a few relevant pieces per query, generate the answer.
- Cost is now proportional to *what you actually need*, not to the size of the corpus. Storage is amortized. Indexes are updatable. The model only ever sees relevant text, so quality and faithfulness improve.

Definition

- **Retrieval Augmented Generation (RAG):**
  A system that, before answering a query, *retrieves* relevant documents from an external knowledge store and supplies them to a language model as additional context for *generation*.
- **Origin:** Lewis, Perez, Piktus, et al. (2020), *"Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,"* NeurIPS.
- Originally framed as a research architecture for open-domain QA. Now the *de facto* production pattern for any LLM application that touches a custom knowledge base: customer-support bots, legal research, medical Q&A, code search, and academic research.
- **Key idea (worth repeating):** The LLM is the *reasoner*. The vector store is the *memory*. The retriever is the *librarian*. RAG is how we connect them.

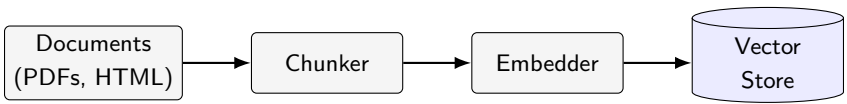
End-to-End Architecture



Five components:

1. **Embedder** — maps text to vectors (Section V).
2. **Vector store** — holds embeddings of every chunk (Section VI).
3. **Retriever** — pulls the top-K closest chunks to the query.
4. **Augmented prompt** — retrieved chunks + user question + instructions (Section VII).
5. **Generator** — the LLM that writes the final answer.

Two Phases — Offline Indexing



- Done **once** — or incrementally as new documents arrive.
- Cost is paid up front; queries are then cheap.
- **Library catalog analogy:** Building the card catalog is slow work. Once it exists, looking things up is fast.
- **For an economist:** Index the entire FOMC archive (1993–present) one weekend; then query it for years.

Two Phases — Online Query

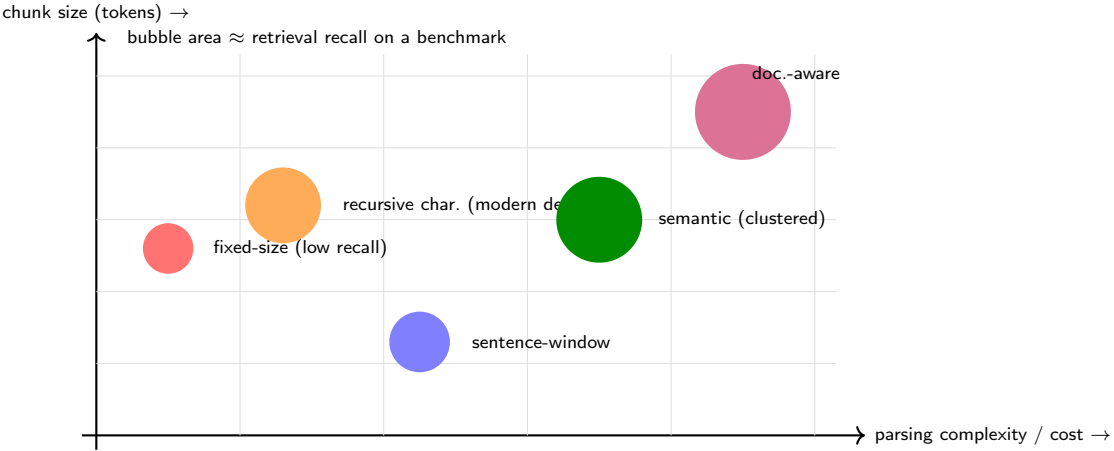
- Done **every time the user asks a question**.
- Each query embeds once, retrieves a handful of chunks, and pays for only those tokens at the LLM stage.
- **Ask-the-librarian analogy:** You walk up with a question, the librarian hands you 5 relevant pages, you read them and answer.
- **Use the same embedder for indexing and querying** — otherwise the geometry doesn't line up.

Chunking

Why Chunk At All?

- Documents are too big to embed as a single vector. We must split them into smaller pieces — *chunks* — before indexing.
- **Three reasons to chunk:**
  1. **Embedder input limits.** Most embedding models accept 512–8192 tokens per input. A 60-page FOMC transcript blows past every limit.
  2. **Granularity of relevance.** If you index a whole 30-page document as one vector, the embedding averages over many topics. A query about "inflation expectations" may match a document that contains *one paragraph* on the topic — but the average vector won't reflect it.
  3. **Generator context budget.** You only inject a few chunks into the LLM prompt. Smaller chunks $\Rightarrow$ more chunks fit $\Rightarrow$ richer evidence base.
- **Trade-off:** Chunks too large $\Rightarrow$ noisy embeddings. Chunks too small $\Rightarrow$ context lost. *Choosing chunk size is the most consequential decision in a RAG pipeline.*

Chunking Strategies — The Trade-Off, Visually



Practical recommendation: start with *recursive character* (LangChain default — big orange bubble, low setup cost). Move *up the y-axis to document-aware* when the corpus has rich structure (FOMC, 10-K, Congress). Reach for *semantic* only after you have measured retrieval quality and can justify the

Fixed-Size vs. Recursive — An Example

- **Source paragraph** (FOMC minutes, simplified):
  "Participants observed that inflation remained elevated. Several noted that core services prices, especially shelter, continued to rise. A few participants expressed concern about persistence in wage growth. Most agreed that policy should remain restrictive."

- **Fixed 60-character chunks:**
  [■Participants observed that inflation remained elevat■]
  [■ed. Several noted that core services prices, especi■]
  [■ally shelter, continued to rise. A few participants■] ...

  ⇒ **Cuts mid-word.** Embeddings of these fragments are nearly useless.

- **Recursive chunking** (split on paragraph → sentence → word, target ∼120 chars):
  [■Participants observed that inflation remained elevated.■]
  [■Several noted that core services prices, especially shelter, continued to rise.■]
  [■A few participants expressed concern about persistence in wage growth.■]

  ⇒ Each chunk is a self-contained sentence. Embeddings carry meaning.

Semantic Chunking

- **Idea:** Don't fix chunk size in advance. Instead, embed each sentence, measure similarity between adjacent sentences, and split where the similarity *drops* — i.e., where the topic changes.
- **Algorithm sketch:**
  1. Split the document into sentences.
  2. Embed each sentence with a fast embedder.
  3. Compute cosine similarity between consecutive sentence embeddings.
  4. Mark a chunk boundary wherever similarity falls below a threshold (or a percentile cutoff).
- **Pros:** Each chunk is topically coherent — you don't slice mid-thought.
- **Cons:** Costs an embedding pass over every sentence *at indexing time*. For very large corpora this is non-trivial.
- **When to use:** When recursive chunking gives noisy retrieval and you have evidence that within-document topic shifts matter (long technical reports, multi-section speeches).

Practical Defaults + Code

- **Sensible defaults** for institutional text:
  - Chunk size: 500–1000 tokens (∼2–4 paragraphs)
  - Chunk overlap: 10–20% (so a sentence on a chunk boundary appears in both neighbors)
  - **Always preserve metadata**: doc ID, page number, section heading, date, author
- **LangChain recursive chunker** (the de facto standard):

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,            # ~600 tokens
    chunk_overlap=100,         # ~80 tokens of overlap
    separators=["\n\n", "\n", ". ", " ", ""],
)

chunks = splitter.create_documents(
    texts=[fomc_minutes_text],
    metadatas=[{"meeting": "2024-06", "doc_type": "minutes"}],
)
```

The Economist's Chunking Gotcha

- Real institutional documents have **structure** that naive chunkers destroy.
- **FOMC minutes:** organized as *speaker turns*. "Participants generally agreed. . . ", "A few members noted. . . ". Cutting across speaker boundaries fuses unrelated viewpoints.
- **Congressional Record:** hierarchical metadata — Congress / session / chamber / date / speaker / bill. Lose the hierarchy and you can't filter or cite properly.
- **SEC 10-K filings:** standardized item numbers (Item 1, Item 1A Risk Factors, Item 7 MD&A). A query about "risk factors" should retrieve only *Item 1A* content — which only works if you parse the structure first.
- **Practical fix:** Write a corpus-specific parser that emits chunks with rich metadata. Extra effort up front; massive payoff in retrieval quality and ability to filter.
- **Lesson:** Don't reach for the generic chunker. Let your corpus tell you what a chunk should be.

Embeddings

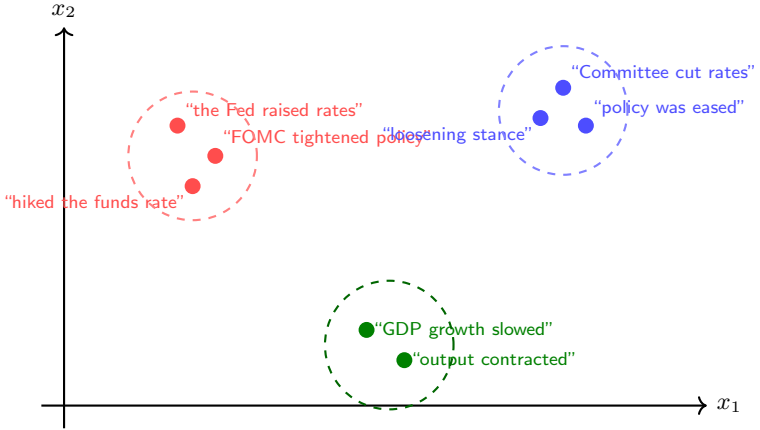
From Text to Vectors — What and Why

- An **embedding** is a function $E : \text{text} \to \mathbb{R}^d$ that maps a piece of text to a fixed-length vector of real numbers.
- Typical dimensionalities: $d \in \{384, 768, 1024, 1536, 3072\}$.
- **The crucial property:**

$$\text{semantically similar texts} \implies \text{nearby vectors in } \mathbb{R}^d$$

- **Why this is useful:** Once your corpus lives in a vector space, "find documents about inflation expectations" becomes a *geometric* operation: compute the embedding of the query, find its nearest neighbors among the document embeddings.
- **Recap from Lec. 7:** Embeddings appear *inside* the transformer (token embeddings). Here we use the *sentence-* or *document-level* embedding produced by a separate retrieval-tuned model.

Geometric Intuition



Reading the picture:

- Hawkish-tightening sentences cluster on the left.
- Dovish-easing sentences cluster on the right.

Why This Works — Pooling + Contrastive Loss (Callback to Lec. 7 T2b)

- **The transformer outputs one vector per token.** A *retrieval embedder* pools those token vectors into a single *document vector* (mean-pool, `[CLS]`, or a learned head). See Lec. 7 T2b "Token Embeddings vs. Document Embeddings" for the geometric derivation.
- **The pooled embedder is trained with a contrastive loss** on *(query, positive, negative)* triples: pull semantically related text together, push unrelated text apart. The geometry is *learned for similarity*, not inherited from the next-token objective.
- **Modern retrieval embedders** (`bge-large-en`, `voyage-3`, OpenAI `text-embedding-3`, Sentence-BERT lineage from Reimers & Gurevych, 2019) all share this pattern. Differences are scale of training data and pooling head, not the core idea.
- **Take-away:** an embedder is a transformer encoder *plus* a pooling head *plus* a contrastive objective. Lec. 7 covered the encoder; Lec. 8 wires in the rest.

Modern Embedding Models — The Four You'll Actually Pick

Model	Dim	Cost	Hosting	Why pick it
OpenAI `text-embedding-3-small`	1536	$0.02/M	API	cheap default, no infra
OpenAI `text-embedding-3-large`	3072	$0.13/M	API	top accuracy, paid
`voyage-3`	1024	$0.06/M	API	top retrieval benchmarks
`bge-large-en-v1.5`	1024	free	local	best open-source local default

Model facts (dimensions and prices) current as of June 2026.

For an academic researcher:

- **Local + free:** `bge-large-en-v1.5`. Runs on a laptop GPU; competitive with paid APIs.
- **Cloud + simple:** OpenAI `text-embedding-3-small`. Cheap, no infra.
- **Top accuracy + budget:** `voyage-3` or OpenAI `3-large`.

Full 7-row reference table (incl. Cohere, MiniLM, E5-Mistral) is in the deck appendix.

Economists' Embedding Example

- **The setup.** Take every Federal Reserve Chair speech from 2010 to 2025 (Bernanke, Yellen, Powell). Embed each speech with `bge-large-en-v1.5`. Project to 2D with UMAP. Plot.
- **What you see:**
  - Crisis-era speeches (Bernanke 2010–2014) cluster together — focused on QE, financial stability.
  - Normalization-era speeches (Yellen 2015–2017) form a distinct cluster — gradual exit, forward guidance.
  - Pandemic and post-pandemic Powell (2020–2025) splits into two: 2020–2021 dovish "patient" speeches vs. 2022–2024 hawkish "restrictive" speeches.
- **Why this matters for research.** The clustering is generated *purely from the text*, with no labels. The economist now has a reproducible measure of stance shifts that can be linked to interest-rate decisions, market reactions, or macro outcomes.
- **Real papers using this approach:**
  - Hansen, McMahon & Prat (2018), *QJE*: text analysis of FOMC transcripts.
  - Shapiro & Wilson (2021), *Review of Economics and Statistics*: Fed sentiment from speeches.
  - Gentzkow, Kelly & Taddy (2019), *JEL*: text as data, survey.

The Domain-Adaptation Problem

- Generic embedders are trained on the open web (Wikipedia, Common Crawl, books, code). They handle ordinary English well.
- **They struggle with economics jargon:**
  - "*patient*" in a Fed statement = *dovish* (a domain-specific code word).
  - CUSIP codes, BEA NIPA tables, FRED series IDs — alphanumeric strings with no semantic content for a generic embedder.
  - Phrases like "Section 13(3)" or "LSAP" (Large-Scale Asset Purchases) are technical citations the model has never seen.
- **Two fixes:**
  1. **Fine-tune the embedder** on a domain corpus. Cheap (a few GPU-hours), big quality gains. Tools: `sentence-transformers` `MultipleNegativesRankingLoss`.
  2. **Hybrid retrieval** (Section VI). Combine dense embeddings with classical sparse retrieval (BM25), which catches exact-match queries the embedder misses.
- **In practice:** Hybrid is usually cheaper and gets you 80% of the gain. Reach for fine-tuning only when retrieval quality is your bottleneck.

Code: Embed FOMC Chunks

```python
from sentence_transformers import SentenceTransformer

# 1. Load a strong open-source embedder
model = SentenceTransformer("BAAI/bge-large-en-v1.5")

# 2. Suppose `chunks` is a list of FOMC chunk strings
chunks = [
    "Participants observed that inflation remained elevated.",
    "Several noted that core services prices, especially shelter, continued
        to rise.",
    "Most agreed that policy should remain restrictive.",
    # ... thousands more
]

# 3. Embed -- one numpy vector per chunk
embeddings = model.encode(
    chunks,
    batch_size=32,
```

Hands-On Checkpoint: The Wiki Demo

- You now know **chunking** (Section IV) and **embeddings** (Section V). Before T3 layers on vector databases, try the simplest possible end-to-end pipeline.
- **The `wiki_demo/` in this lecture folder** ships a tiny corpus and a *TF-IDF* retriever (no vector DB, no API key, no network):

```
cd Lec08_RAG/wiki_demo
ls knowledge/             # browse the seeded articles
# point an LLM at index.md and ask a question; it reads the
# wiki and answers from it (Karpathy-style pre-formal RAG).
```

- **Why this checkpoint matters:** the wiki demo is the simplest version of the pipeline you've just learned. Chunks are *markdown articles*; "embeddings" are TF-IDF keyword vectors; the retriever is exact-match lookup. Replace any one of these with the modern equivalent and you have RAG.
- **Extension exercise.** Drop one of your own working papers into `knowledge/`, regenerate the index, and ask a question. If retrieval surfaces the relevant article, you have shipped your first single-user RAG.

Topic 8.2 Wrap-Up

- **We now have a numbered, vectorized corpus.** Chunks carry metadata; embeddings give them geometry.
- **Three load-bearing decisions:** chunk strategy (recursive vs document-aware), embedder choice (open-source vs API), domain adaptation (hybrid first; fine-tune the embedder only if retrieval bottlenecks).
- **Next (Topic 8.3):** where do the vectors live? Vector databases, ANN search, hybrid sparse+dense retrieval, re-ranking, and a first taste of GraphRAG for relationship questions.

Appendix — Modern Embedding Models (Full 7-Row Reference)

Model	Dim	Cost	Hosting	Notes
OpenAI `text-embedding-3-small`	1536	$0.02/M	API	cheap, fast, solid
OpenAI `text-embedding-3-large`	3072	$0.13/M	API	top-tier accuracy
Cohere `embed-v3`	1024	$0.10/M	API	strong on English
`voyage-3`	1024	$0.06/M	API	top retrieval benchmarks
`all-MiniLM-L6-v2`	384	free	local	tiny, very fast, weaker
`bge-large-en-v1.5`	1024	free	local	top open-source
`e5-mistral-7b-instruct`	4096	free	local (GPU)	frontier open model

Model facts (dimensions and prices) current as of June 2026.

The compressed 4-row table in the main flow keeps only the models a researcher would realistically pick on day one. Cohere, MiniLM, and E5-Mistral are recorded here for completeness and as escape hatches when the four defaults underperform.